SIMATS ENGINEERING

Department of Computer Science and Engineering

Course Code:	CSA1512	**Assessment:**	CO5 AT1
Course Name:	Cloud Computing and Big Data Analytics	**Type:**	Constraint-Based Problem
CO Statement:	Apply big data storage, integration, Hadoop, HDFS, and MapReduce concepts for distributed data processing		

Part A: HDFS Fundamentals – Lesson Notes

What is HDFS?

The Hadoop Distributed File System (HDFS) is a distributed storage system designed to reliably store very large files (hundreds of MB or more) across multiple machines, to stream those files at high bandwidth, and to tolerate node failures gracefully. Key design principles:

1. Fault tolerance through replication – files are replicated across multiple nodes.
2. Write-once-read-many access – simplifies data consistency.
3. High throughput over latency – optimized for batch processing, not real-time.

Lesson Note: HDFS vs. Traditional File Systems

HDFS is NOT a general-purpose file system like ext4 or NTFS. It trades random access and POSIX compliance for high throughput on large files and fault tolerance. A single HDFS file might be 100GB+, split into blocks across dozens of DataNodes.

HDFS Architecture

An HDFS cluster consists of three main components:

Component	Role	Count per Cluster
NameNode	Manages file system namespace; maintains file system tree + metadata for files/folders. Does NOT store actual data.	1 (or 1 primary + 1 secondary for HA)
Secondary NameNode	NOT a backup NameNode. Performs housekeeping (merges namespace image + edit logs). Prevents NameNode's edit logs from becoming too large.	1
DataNodes	Store the actual data blocks. Perform block creation, deletion, replication on instruction from NameNode.	Many (typically 10–100s)

Critical Insight: Single Point of Failure

The NameNode holds the entire file system namespace in RAM. If it crashes, the cluster is unusable until restart. This is why a Secondary NameNode (or HA pair) is essential in production.

Blocks & Replication

HDFS breaks each file into blocks (default 128MB or 256MB). Each block is replicated across multiple DataNodes. The replication factor (RF) controls how many copies exist.

Replication Factor	Fault Tolerance	Storage Overhead	Typical Use Case
1	None – single node failure loses data	100%	Non-critical data, dev/test
2	Survive 1 rack failure (if nodes on same rack); single node failure OK	200%	Development clusters
3	Survive 1 rack failure + 1 node in another rack	300%	Production (Hadoop default)

Rack Awareness Strategy

With RF=3: place 2 replicas on same rack (fast write); 1 replica on different rack (survival of rack-level disaster). This balances write speed (same rack) vs. fault tolerance (different racks).

Block Sizing & Data Locality

- **128MB / 256MB default block size:** Prevents NameNode RAM exhaustion by minimizing total block metadata count.
- **Locality Tiers:** Node-local (Map task scheduled on the node containing block) → Rack-local (Map task scheduled on the same rack switch) → Off-rack (Data transferred over core switches).

Part B: Main Scenario – Web Log Analysis Cluster

Problem Statement

Scenario: A company operates 100 web servers generating web access logs. Each server generates ~5GB of logs per day. The company needs to:

- Store 2 years of logs (36.5TB raw data)
- Run daily MapReduce jobs to analyze visitor patterns
- Survive single rack failure without data loss
- Minimize hardware costs while ensuring high availability

Lesson Note: Constraint-Based Design

Real-world systems have conflicting constraints: high availability wants more replication (higher cost); cost minimization wants less replication. Your job is to balance them under business constraints.

Step 1: Calculate Storage Requirements

Instructions: Fill in the worksheet below.

Metric	Value	Your Answer
Raw data (100 servers × 5GB/day × 365 × 2 years)	36,500 GB = 36.5 TB	36,5000 GB(36.5TB)
Replication factor assumed	3× (survive 1 rack fail)	3
Total raw disk needed (raw data × RF)	$36.5 \text{ TB} \times 3$	109.5TB
Overhead for metadata (~5% of raw)	$109.5 \text{ TB} * 0.05$	5.475 TB (~5.5 TB)
Usable capacity after overhead	$109.5 \text{ TB} + 5.5 \text{ TB}$	114.975 TB (~115 TB)

Guided Calculation

Total raw disk = 36.5 TB × 3 = 109.5 TB
Metadata overhead ≈ 109.5 × 0.05 = 5.5 TB
Usable capacity ≈ 109.5 + 5.5 = 115 TB

Step 2: Size the DataNode Hardware

Typical DataNode specs: 12TB disk, 64GB RAM. Design the cluster:

Item	Calculation	Your Answer
Number of DataNodes needed	115 TB ÷ 12 TB/node = ~10 nodes	10 NODES
Total RAM across cluster	10 nodes × 64 GB = 640 GB	640 GB
NameNode heap size (rule: 1GB per 1M files)	If ~50M files → 50GB NameNode heap	4 GB to 8 GB JVM Heap

Step 3: Replication & Rack Topology

Design the rack layout. Assume 2 racks (each can hold ~5 nodes).

Topology Layout

Rack 1: 5 DataNodes (DN1–DN5) Rack 2: 5 DataNodes (DN6–DN10) NameNode + Secondary NameNode: Separate machine (NOT on a DataNode).

Fill in the replication strategy:

Replica #	Placement	Rationale
1st replica	Local node in **Rack 1** (e.g., DN1)	Maximizes write pipeline ingest speed and guarantees local write efficiency.
2nd replica	Remote node in **Rack 1** (e.g., DN2)	Minimizes cross-rack switch network saturation during write replication pipelines.
3rd replica	Remote node in **Rack 2** (e.g., DN6)	SD

Step 4: Fault Tolerance Analysis

Scenario: Rack 1 fails (all 5 DataNodes go down).

Instructions: Analyze data loss for different replica placements.

Expected Insight

If all 3 replicas are in Rack 1, losing Rack 1 means total data loss. The standard strategy spreads replicas: 2 in one rack, 1 in another, preventing rack-level loss.

Step 5: MapReduce Job Scheduling

A daily MapReduce job processes the logs: input = 5GB/day × 2 years = 36.5TB.

Metric	Calculation	Your Answer
Block size	128 MB (default)	128MB
Number of blocks	36.5 TB ÷ 128 MB = ~292,000 blocks	299,008 Blocks (~292,000–299,000)
Number of mappers (≈ 1 per block)	~292,000 mappers	~299,008 Map Tasks
Data locality %	If 3 replicas well-distributed, expect 90%+ node-local	≥ 90% Node-Local.

Step 6: Cost-Benefit Analysis

Calculate total cost. Assume: $500 per DataNode (hardware + network), $1000 annual per DataNode (power, cooling, ops).

Cost Item	Calculation	Total
Initial hardware (10 DataNodes)	10 × $500	$5,000
Annual operations (10 DataNodes)	10 × $1000	$10,000 / year
2-year total cost	($5,000 (hardware) + (2 × $10,000)	$25,000
Cost per TB stored (2 years)	$25,000/36.5 TB	$684.93 / TB

Trade-off Question:

If you reduce RF from 3 to 2, you halve storage costs but lose rack-fault tolerance. Is this acceptable? Justify.

Reducing RF to 2 is **unacceptable** under the stated SLA constraints. With only two racks, RF=2 would require placing 1 replica per rack. While surviving a single node failure, any concurrent drive loss during a rack maintenance outage results in unrecoverable raw data corruption.

Part C: Mini-Problems

Mini-Problem 1: Scaling for Peak Traffic

In Month 6, the company adds 50 new web servers (150 total), doubling log generation to 10GB/server/day. How does the HDFS cluster scale?

• New annual raw data: 150 servers × 10GB/day × 365 days = 547.5 TB/year (at 2-year scale: 1095 TB)

Your task: Calculate new DataNode count, storage costs, and replication strategy.

Data Growth: 150 servers $\times$ 10 GB/day $\times$ 365 days $\times$ 2 years $=$ $1{,}095$ TB raw data.
Total Storage Required: $1{,}095$ TB $\times$ 3 (RF) $\times$ 1.05 (overhead) $=$ $3{,}449.25$ TB.
Hardware Sizing: $[3{,}449.25$ TB$/12$ TB per node$]$ $=$ $\mathbf{288}$ DataNodes.
Hardware Capex: $288 \times \$500 = \$144{,}000$.
2-Year Opex: $288 \times \$1{,}000 \times 2 = \$576{,}000$.
Total Cost: $\$\mathbf{720{,}000}$.

Mini-Problem 2: Block Size Tuning

Experiment: What if we use 256MB blocks instead of 128MB?

Implications:

• Fewer blocks → fewer map tasks → less parallelism

• Fewer blocks → less NameNode memory

Estimate: With 256MB blocks, how many mappers for the 36.5TB dataset? Pros and cons?

Mapper Calculation (256MB): $(36.5$ TB $\times$ 1024×1024 MB/TB$)/256$ MB $=$ $\mathbf{149{,}504}$ Mappers (50% reduction).
Pros: Halves NameNode metadata memory footprint; reduces JVM task setup/cleanup overhead.
Cons: Decreases maximum parallel task granularity for smaller query subsets.
Verdict: 256MB is superior for large-scale, long-term sequential web log parsing.

Mini-Problem 3: Failure Recovery Timeline

A DataNode in Rack 1 crashes (hardware failure). HDFS detects missing replicas and re-replicates the blocks to other DataNodes. Estimate the re-replication time:

• DataNode disk: 12 TB

• Network bandwidth (typical): 1 Gbps

How long does it take to re-replicate 12 TB from peer DataNodes?

Data on Failed Node: 12 TB $=$ $96{,}000$ Gb.
Network Speed: 1 Gbps NIC.
Replication Bandwidth Utilization: Dedicated rebuild limit at 80% link capacity (0.8 Gbps).

$$\text{Recovery Time} = \frac{96{,}000 \text{ Gb}}{0.8 \text{ Gbps}} = 120{,}000 \text{ seconds} \approx \mathbf{33.33} \text{ hours}$$

Mini-Problem 4: Compression Trade-offs

Compress the logs with Gzip (typical compression ratio 3:1 for text logs). How does this affect storage and MapReduce job performance?

New compressed raw data: 36.5 TB ÷ 3 = 12.17 TB

• Pro: Storage cost drops by 2/3

• Con: Mappers must decompress on-the-fly (higher CPU)

Decision: Should we compress? Justify based on the numbers.

Compressed Raw Volume: 36.5 TB$/3 = 12.167$ TB.
Required Disk with RF=3: 12.167 TB $\times$ $3 \times 1.05 = \mathbf{38.33}$ TB.
Nodes Required: $[38.33/12] = \mathbf{4}$ DataNodes (Down from 10 nodes).

2-Year Total Cost: $(4 \times \$500) + (4 \times \$1{,}000 \times 2) = \$\mathbf{10,000}$ (Saves $15,000 or 60%).

Decision: Adopt compression. Use container formats like Hadoop Snappy or Splittable Gzip (.deflate index) to balance CPU decompression load while cutting infrastructure cost by 60%.

Part D: Problem Bank – 4 Additional Scenarios

Scenario A: Real-Time Data Warehouse (E-commerce)

An e-commerce platform collects real-time user behavior (click streams, purchases) from mobile and web. Volume: 2TB/day. Need to query data within 1 hour for BI dashboards. Design an HDFS cluster with constraints: (1) RF=2 for cost, (2) 30-node cluster budget, (3) 1-day data retention (rolling 24-hour window). Deliverables: Cluster sizing, replication strategy, block size justification, 24-hour re-replication risk.

Data Sizing: 2 TB/day $\times$ 1 day retention $\times$ 2 (RF) $\times$ 1.05 $=$ $\mathbf{4.2}$ TB usable.
Hardware: 30 nodes available. Run in high-memory, multi-disk configurations to handle micro-batch ingest (Spark Streaming).
Block Size: 64MB to optimize fast batch interval commit frequencies.
Re-replication Risk: Under RF=2, if a node fails, 2.1TB must re-replicate immediately. At 1Gbps, re-replication finishes in under 6 hours, remaining inside the 24-hour retention window.

Scenario B: Backup & Archive (Regulatory Compliance)

A financial services firm must retain transaction logs for 7 years (compliance). Volume: 500GB/day. Budget: minimize cost (data is infrequently accessed). Design an HDFS cluster with: (1) RF=1 for cost, (2) separate cold-storage tier for logs older than 1 year, (3) quarterly backups to off-site. Deliverables: Cost analysis (7-year total), replication strategy, backup plan, data recovery RTO/RPO.

Data Sizing (7 Years): 0.5 TB/day $\times$ 365 $\times$ 7 $=$ $1{,}277.5$ TB raw.
Storage Tiering:

- Year 1 (Active Archive): RF=2 on commodity HDFS (365 TB usable).
- Years 2–7 (Cold Archive): RF=1 with HDFS Erasure Coding (RS-6-3) reducing overhead to $1.5\times$ while ensuring fault tolerance.

Scenario C: IoT Sensor Network (Streaming + Batch)

IoT devices (temperature, humidity, pressure) stream data from 10,000 sensors at 1 reading/min (14.4M readings/day). Data volume: ~2GB/day (small payloads, high frequency). Need to: (1) retain 2 years, (2) detect anomalies in real-time (need fast access), (3) run weekly batch analytics. Deliverables: HDFS cluster design (NameNode heap for millions of small files!), block size & replica count, comparison with time-series DB (InfluxDB, etc.).

Small File Mitigation: 14.4M daily small packets directly crashing NameNode RAM must not be written as individual files.
Ingest Design: Stream sensor events into Apache Kafka → Apache Flume / Spark Streaming → batch write into partitioned Apache SequenceFiles or Apache ORC/Parquet at 128MB block intervals.
NameNode Footprint: Aggregates 14.4M daily records into ~16 files per day (2 GB$/128$ MB), keeping NameNode heap consumption under 10MB/year.

Scenario D: Multi-Tenant Cloud (SaaS)

A SaaS platform provides data analytics to multiple customers. Each customer's data is isolated in separate HDFS directories. Total: 100 customers, 1TB/customer on average. Constraint: (1) RF=3 for HA, (2) each customer must not access another's data. Deliverables: Cluster architecture (single cluster + multi-tenancy or separate clusters?), HDFS permissions & quotas, cost-per-customer estimate, cross-tenant security risks.

- **Architecture:** Shared large-scale HDFS cluster with logically isolated tenant namespaces (`/tenants/tenant_id`).

POSIX Permissions & HDFS ACLs: Restrict access to designated service accounts (chmod 700 /tenants/{tenant_id}).

Evaluation Rubric

Criteria	Wt.	L4 (40–50)	L3 (30–39)	L2 (20–29)	L1 (0–19)
HDFS Concepts Understanding	20%	Accurate understanding of blocks, replication, NameNode role	Mostly correct; minor gaps	Basic understanding; some errors	Misinterprets core concepts
Constraint Analysis	20%	Identifies all constraints (cost, FT, scale); prioritizes well	Most constraints identified; minor prioritization gaps	Some constraints missed or poorly weighted	Ignores constraints or to analyze
Design & Calculations	30%	Correct cluster sizing, RF strategy, topology; justified	Mostly correct; minor arithmetic/logic gaps	Correct approach but calculation errors	Incorrect design or mis major elements
Trade-off & Risk Analysis	20%	Thorough analysis: cost vs. availability; risk mitigation	Good analysis; some trade-offs missed	Basic analysis; limited depth	Little or no trade-off analysis
Documentation & Clarity	10%	Clear diagrams, formulas, step-by-step reasoning	Mostly clear; minor formatting issues	Understandable but lacks detail	Unclear or poorly organized

Total Marks: 100 (20 + 20 + 30 + 20 + 10)